

Pocket Lipid Interaction

Florian Echelard

Summary.

Using Voronota and Pytorch to investigate interactions

1 Introduction

Lipid transfer proteins (LTPs) are Peripheral Membrane Proteins that transfer a lipid from a donor membrane to an acceptor membrane with specificity for each membrane, and for the lipid. Their structure involves an hydrophobic cavity that covers the entire lipid. Lipidomics experiments yielded novel results of LTPs binding to liposomes. Using interaction data detailing the lipid composition of the membrane bound by proteins of this family, we will analyze in detail the properties and mechanisms we can outline from these experiments. As the experiment lay out genes for proteins and profiles for lipid, some preprocessing for selecting the data is necessary. The approach we took is structural for the protein, using Graph Neural Network to interpret features, using a pytorch library that focuses on this [1] [2]. For the lipid we do not have the complex (meaning the protein and lipid bound in one structure), so we use the SMILES 1D encoding as the lipid structure, with a Chemical Language Model specifically trained on small molecules [3].

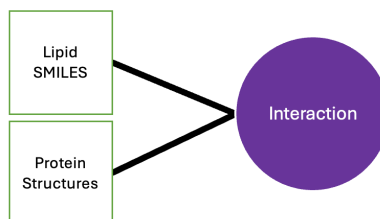


Figure 1: Goal of the study, predict the interaction (yes/no) using two different representations and multimodal learning

2 Data

2.1 Protein Structures

Selected structures without mutations (**data/structures/possible_pdb_numbers**) from the PDB, AFDB if not available. Based on Uniprot Family and Domain annotation (mostly from PROSITE) (**data/structures/raw**)

2.2 Lipid SMILES

Using the description of the lipid head group and chain (**data/LTP-lipid_interaction.csv**) from Mass Spectrometry data. We then submit REST API requests to LipidMaps (**data/lipid_maps_fetch.py**) We get SMILES format back for the molecules, which are one of the ways to annotate 2D graphs in one dimension. [4]

2.3 Interactions

All positive interactions are present in (`data/LTP-lipid_interaction.csv`), each line is a recorded lipid inside of a LTP [5].

3 Preprocessing

Additional information must be generated to identify the pocket and its properties.

3.1 Duplicates

In the original dataset, several lines pertain to the same interaction, but from different experiments. A choice was made to combine profiles with no sidechain description with profiles that do have these details. (`data/Processed_dataset.csv`)

3.2 Negative Interactions

The previous iteration of the training used every LTP-lipid pair possible that is not in the original dataset as a negative interaction. This approach may not be best, especially now considering that we went from 1137 positive and 14000 negative interaction to 751 positive interactions.

3.3 Voronota

Using the Voronota software’s [6] `voronota-pocket` to annotate pockets in the .pdb’s b-factor column (`preprocessing/voronota_parameter_per_ltp.xlsx`) instead of the whole annotated structures (`data/structures/voronota`). This is to verify the annotation. Parameters can be changed depending on the type of pocket to consider. LTPs are known to possess large hydrophobic pockets, which is why it is important to test different sets of parameters that show a good cutoff between the hydrophobic pocket and the grooves around the surface. To select a set of parameter that accommodates this criteria, we use visual inspection on the annotated structures with pymol, to verify that this set is satisfactory for all proteins in our dataset. To generate the graphs used in the training, we use a script from `voronota-js` (`preprocessing/voronota-js-receptor-data-graph`)

These graphs (`data/graphs`) are separated into nodes and edges files, for atom without prefix, and for amino acids with the "coarse" prefix. There are features relevant to the atom and amino acid chemistry, as well as descriptors computed based on the geometry of the protein (see details on : <https://github.com/kliment-olechnovic/generating-graphs-of-protein-receptors>).

There were discussions as to which level of detail is better to train the model. as our current dataset is only composed of 35 protein structures, the atomic level description might be too noisy to construct a good representation. Combining the geometric description of coarse-grained atoms into amino acids is a promising avenue, and this level of definition is important as it is the same dimension for ESM3/PLMs tokens.

3.4 Lipid Embedding

To use the lipid as input, we first need to encode its SMILES into tokens from a pretrained network [3]. We make a list of our SMILES (`embed_smiles.py`). Here we use Molformer, and use a modified script to encode our data (`preprocessing/frozen_embedding.py`) inside of the

git directory (https://github.com/IBM/molformer/tree/main/notebooks/pretrained_molformer). The dimension of a SMILES embedded sequence is $[N, 768]$. An additional token is added at the beginning of the sequence, the $<CLS>$ token, that can be used instead of pooling on the whole sequence, as it was used in training to represent the whole sequence with attention.

3.5 Protein Embedding

The voronota graph script features a lot of aspects of the structure, but during tests, we weren't able to learn any task by only using geometric and chemical descriptors. So by coarsegraining on amino acids and using Protein Language Model embeddings of the sequence and projecting it on the structure, results were a bit better. So we use ESM3 [7] to embed these sequences (**preprocessing/EmbedProtein.py**). The current available model for generating embeddings based on the sequence is "esm3-sm-open-v1" (<https://huggingface.co/EvolutionaryScale/esm3-sm-open-v1>).

The embeddings are accessible in the (**data/embedding_ESM3**) directory, compared to the coarse-grained residue nodes of the graph, the model adds two tokens, a $<BOS>$ at the beginning and $<EOS>$ at the end. To make sure the token dimensions matches with the coarse grained graph nodes, it is better to remove the before concatenating them with Voronota descriptors. The dimension of a sequence is $[N, 1536]$ so to keep the relative importance of other features the latent dimension should be projected by a linear layer before concatenation with other features.

4 Achitecture

4.1 Lipid

As we are using a pre-trained network embedding as our input, we can use feed forward or self-attention layers to train our representation. We consider the lipid as tokens within a graph in our approach, but we do not have specific edges, so it is initially considered fully connected. To test this part of the architecture, we have a task to predict a head group from the embedding. (**architecture/toy_task/LipPred.py**)

4.2 Protein

The protein graph is sparse, and we have geometric and chemical features for every atom, which can be coarse grained into amino acids. We can then combine embeddings from PLMs. To create a representation for the protein, we can use a combination of Graph Convolutional Networks and Graph Attention Network. To test this part of the architecture, we can predict the domain type the structure is part of. (**architecture/toy_task/ProtPred.py**)

4.3 Cross Attention

To have the two representations communicate and predict a property, we use Cross Attention. As multimodal networks implementation depends on the data used, several approaches and combination of data have to be tested.

The original design of the cross attention relied on an antibody interaction deep learning model, that uses multimodal input and a combination of cross and self attention to create a representation to update the relative position of the complex (https://www.mlsb.io/papers_2022/End_to_end_accurate_and_high_throughput_modeling_of_antibody_antigen_complexes.pdf).

5 Training

5.1 Loss

As the prediction for the interaction is binary, Binary Cross Entropy Loss seems the most appropriate.

5.2 Data Split

Several approaches are possible for data splitting. The domain types are unevenly distributed, there is less protein diversity than lipids. So we can approach splitting the train, validation, and test by looking at the domain family of the protein, or the head group of the lipid. If we want to test for generalization, we can leave out a whole class of proteins or lipid, as their representation alone should allow to extrapolate on new data. An approach of randomised sampling will very probably encompass all the proteins, but can sideline some lipids. This can be interesting for early tests.

Keep in mind that the dataset reveals only positive interactions. The negative interactions are generated based on the rule that if a lipid was not seen as transported by a certain protein in the experiments, it's that it doesn't. This assumption brings about a huge bias, considering that you may want to restrict negative interactions to a comparative number to the positive one during training. It means that based on your sampling of "negative" interactions, the model will yield different results.

Another bias of unknown impact is the fact that the original dataset does not include detailed structure for the lipid, meaning that most of the time the structure of the chains is uncertain, both their carbon length, and the localisation of the unsaturation. It means that the SMILES used are similar at best to the actual molecule seen during mass spectrometry experiments, but every data point probably has slight difference from the ground truth in some of the aspects described here. Which cannot be addressed unless additional experiments are made for all the systems and interactions.

The batch object comes from `torch_geometric.data.Data()`, both for the protein graph and the lipid. This allows to batch several graphs together by using an adjacency matrix.

References

- [1] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. **PyTorch: An Imperative Style, High-Performance Deep Learning Library**. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [2] Matthias Fey and Jan E. Lenssen. "Fast Graph Representation Learning with PyTorch Geometric". In: **ICLR Workshop on Representation Learning on Graphs and Manifolds**. 2019.
- [3] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. "Large-Scale Chemical Language Representations Capture Molecular Structure and Properties". In: **bioRxiv** (2022). arXiv: 2106.09553 [cs.LG]. URL: <https://arxiv.org/abs/2106.09553>.

- [4] Matthew J Conroy, Robert M Andrews, Simon Andrews, Lauren Cockayne, Edward A Dennis, Eoin Fahy, Caroline Gaud, William J Griffiths, Geoff Jukes, Maksim Kolchin, Karla Mendivelso, Andrea F Lopez-Clavijo, Caroline Ready, Shankar Subramaniam, and Valerie B O'Donnell. "LIPID MAPS: update to databases and tools for the lipidomics community". In: **Nucleic Acids Research** 52.D1 (Oct. 2023), pp. D1677–D1682. ISSN: 0305-1048. DOI: 10.1093/nar/gkad896. eprint: <https://academic.oup.com/nar/article-pdf/52/D1/D1677/55040242/gkad896.pdf>. URL: <https://doi.org/10.1093/nar/gkad896>.
- [5] Kevin Titeca, Antonella Chiapparino, Dénes Türei, Joanna Zukowska, Larissa van Ek, Mahmoud Moqadam, Sergio Triana, Inger Ødum Nielsen, Charlotte Gehin, Kenji Maeda, Theodore Alexandrov, Julio Saez-Rodriguez, Nathalie Reuter, Marco L. Hennrich, and Anne-Claude Gavin. "A system-wide analysis of lipid transfer proteins delineates lipid mobility in human cells". In: **bioRxiv** (2023). DOI: 10.1101/2023.12.21.572821. eprint: <https://www.biorxiv.org/content/early/2023/12/22/2023.12.21.572821.full.pdf>. URL: <https://www.biorxiv.org/content/early/2023/12/22/2023.12.21.572821>.
- [6] Kliment Olechnovič and Česlovas Venclovas. "Voronota: A fast and reliable tool for computing the vertices of the Voronoi diagram of atomic balls". In: **Journal of Computational Chemistry** 35.8 (2014), pp. 672–681. DOI: <https://doi.org/10.1002/jcc.23538>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/jcc.23538>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jcc.23538>.
- [7] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. "Simulating 500 million years of evolution with a language model". In: **bioRxiv** (2024). DOI: 10.1101/2024.07.01.600583. URL: <https://doi.org/10.1101/2024.07.01.600583>.